

UNIX WARMUP

Created by <Academia de Código_>

Road Map:

1. UNIX/Shell theory
2. Meet the bootcamp server
3. Change <Code Cadet> user account password
4. UNIX filesystem
5. Meet the shared folder
6. UNIX environment
7. Meta Characters
8. Which
9. Shell Initialization
10. Filesystem Security

WHAT IS UNIX?

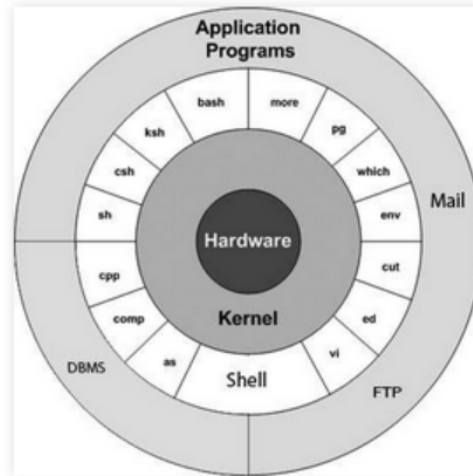
UNIX is a family of operating systems. An operating system is a set of programs that act as a link between the computer and the user.

Of all the computer programs, the subset that allocates the system resources and coordinates all the details of the computer's internals, is called the kernel.

Users **communicate with the kernel** through a program known as the shell. The shell is a command line interpreter; it translates commands entered by the user and converts them into a language that is understood by the kernel.

UNIX STRUCTURAL DIAGRAM

Here is a basic block diagram of a UNIX system



UNIX ARCHITECTURE ELEMENTS

- **Kernel:** The kernel is the heart of the operating system. It interacts with hardware and most of the tasks like memory management, task scheduling and file management.
- **Shell:** The shell is the utility that processes your requests. When you type in a command at your terminal, the shell interprets the command and calls the program that you want. The shell uses standard syntax for all commands. C Shell, Bourne Again Shell and Korn Shell are most famous shells which are available with most of the Unix variants.
- **Commands and Utilities:** There are various commands and utilities which you would use in your day to day activities. **cp**, **mv**, **cat** and **grep** etc. are few examples of commands and utilities. There are over 250 standard commands plus numerous others provided through 3rd party software. All the commands come along with various optional options.
- **Files and Directories:** All data in UNIX is organized into files. All files are organized into directories. These directories are organized into a tree-like structure called the filesystem.

REMOTE SERVER CONNECTION

Connect to your campus' server (aka *campusname.codecadets*). Your account username is the concatenation of your first and last names, and the password will be provided by your Master Coders.

```
$ ssh nunocampelo@lisboa.codecadets.academiadecodigo.org
```

```
      ZZZ      ZZ
     ZZ ZZ    ZZ ZZ
    ZZZZ ZZZZ  ZZZ ZZ
   ZZZ ZZ ZZ  ZZ ZZ
  ZZ ZZ ZZ ZZ ZZ ZZ
 ZZ ZZ ZZ ZZZZZ ZZ
  ZZZ      ZZZ ZZZZ ZZZ
   Z      Z   ZZ ZZ
                ZZ ZZ
                 ZZZZ
                  ZZ
```

```
Welcome to the <Academia de Código > challenge server, have fun!
```

```
Last login: Fri Jan 8 19:02:31 2016 from 62.28.12.174
```

Use **logout** to return to a local shell session:

```
$ logout
```

```
Connection to lisboa.codecadets.academiadecodigo.org closed.
```



UNIX ACCOUNTS

UNDERSTANDING USER ACCOUNT MANAGEMENT

LOGGED USERS

Connect to the bootcamp server:

```
$ ssh campelo@lisboa.codecadets.academiadecodigo.org
```

Find shell currently logged user (who am I):

```
$ whoami  
campelo
```

Find the users logged in the computer at this time:

```
$ users  
campelo mafaldacarvalho  
  
$ who  
campelo pts/0 Jan 9 23:42 (81.84.14.82)  
mafaldacarvalho pts/1 Jan 9 23:43 (81.84.14.82)  
  
$ w  
23:56:18 up 5 days, 13:45, 2 users, load average: 0.00, 0.01, 0.05  
USER TTY FROM LOGIN@ IDLE JCPU PCPU WHAT  
campelo pts/0 81.84.14.82 23:42 0.00s 0.00s 0.00s w  
mafaldac pts/1 81.84.14.82 23:43 12:41 0.00s 0.00s -bash
```



CHANGE PASSWORD

All UNIX systems require passwords to help ensure that your files and data remain your own and the system itself is secure from malicious users.

Change the currently logged user's password with **passwd**:

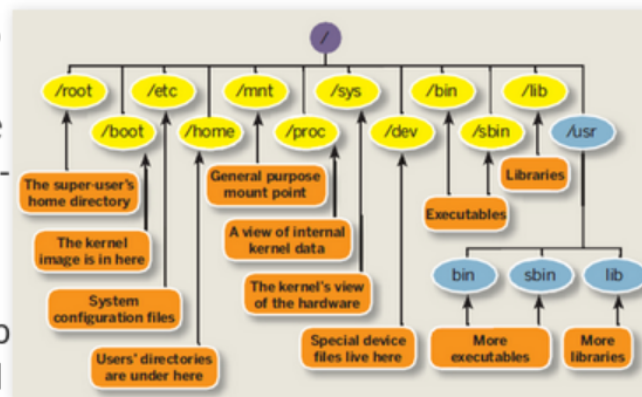
```
$ passwd
Changing password for campelo.
(current) UNIX password:
Enter new UNIX password:
Retype new UNIX password:
passwd: password updated successfully
```

UNIX FILESYSTEM

FILE MANAGEMENT 101

All data in UNIX is organized into files. All files are organized into directories. These directories are organized into a hierarchical tree-like structure called the filesystem.

A directory is a file whose sole job is to store file names and related information. All files, whether ordinary, special, or directories, are contained within a directory.



ROOT DIRECTORY

Directories are arranged in a hierarchy with root (/) at the top.

Navigate to the root folder of the bootcamp server using **cd**:

```
$ cd /
```

Check where you are within the filesystem hierarchy at any time with **pwd**:

```
$ pwd  
/
```

Try listing the files within the root folder with **ls**:

```
$ ls  
bin    dev    home    lib     lost+found  mnt  proc  run  srv  tmp  var  
boot  etc    initrd.img  lib64  media      opt  root  sbin  sys  usr  vmlinuz
```

HOME DIRECTORY

The directory in which you find yourself when you first login is called your home directory.

You will be doing much of your work in your home directory and subdirectories that you'll be creating to organize your files.

Navigate back to your home folder:

```
$ cd ~
```

ABSOLUTE/RELATIVE PATHNAMES

Again, directories are arranged in a hierarchy tree with root (/) at the top. The position of any file within the hierarchy is described by its pathname.

Elements of a pathname are separated by a /. A pathname is absolute if it is described in relation to root, so **absolute** pathnames always begin with a /.

Here are some examples of absolute pathnames:

```
/etc/apache2/apache2.conf  
/var/www/warmup/
```

A pathname can also be relative to your current working directory.

Relative pathnames never begin with /. Relative to campelo's home directory, some pathnames might look like this:

```
Development/warmups  
codecadets/resources/unix-tutorial.txt
```

CREATING DIRECTORIES

In the bootcamp server, within your home directory, create a new folder named warmup:

```
$ mkdir warmup
```

The **mkdir** command produces no output if it successfully creates the requested directory.

Sometimes when you want to create a directory, its parent directory or directories might not exist. In this case, mkdir issues an error message as follows:

```
$ mkdir temp/resources/  
mkdir: cannot create directory 'temp/resources/': No such file or directory
```

In such cases, you can specify the **-p** option to the **mkdir** command. It creates all the necessary directories for you:

```
$ mkdir -p temp/resources/
```



REMOVING DIRECTORIES

Use **rmdir** to remove the empty directory temp/resources:

```
$ rmdir temp/resources
```

With **rmdir** (and **mkdir**), you can remove (create) multiple directories at the same time as follows:

```
$ mkdir development code-breaks temp-folder  
$ rmdir code-breaks
```

MOVING/RENAMING FILES

The **mv** (move) command can be used both to move or rename files (and directories in particular).

You will be renaming/moving a file depending on if you use **mv** to the same/different parent folder.

Rename the temp directory to tmp:

```
$ mv temp tmp
```

LISTING FILES

Use the **ls** command to list the files of your home folder:

```
$ ls ~/
codecadets tmp unixstuff warmup
```

The **ls** command supports the **-l** option which will give you more information about the listed files:

```
$ ls -l ~/
drwxr-xr-x 2 campelo campelo 4096 Jan  8 18:35 codecadets
drwxr-xr-x 2 campelo campelo 4096 Jan 10 12:27 tmp
drwxr-xr-x 2 campelo campelo 4096 Jan  6 19:10 unixstuff
drwxr-xr-x 2 campelo campelo 4096 Jan 10 12:11 warmup
```

Column	Meaning		
1	File type and permissions	5	Bytes size of the file
2	Number of memory blocks taken by the file	6	Creation (modification) date and time
3	The owner of the file (who created it)	7	File name
4	The group of the owner		



METACHARACTERS

Metacharacters are a group of characters that have special meanings to the UNIX operating system. For instance, ***** represents 0 or more characters, and **?** represents (any) single character.

Symbol	Meaning
*	Matches any number of any characters
?	Matches any single character
\ <i>char</i>	Ignore shell's special meaning of <i>char</i>
<i>command</i> < <i>file</i>	Input the content of <i>file</i> to the <i>command</i>
<i>command</i> > <i>file</i>	Send the output of <i>command</i> to the end of the <i>file</i>
<i>command_1</i> <i>command_2</i>	Sends the output of <i>command_1</i> as the input of <i>command_2</i>



THE . & .. DIRECTORIES

The . (dot) filename represents the current working directory; and the .. (dot dot) filename represents the directory one level above the current working directory, often referred to as the parent directory.

Every directory on the UNIX filesystem contains both the . and the .. filenames.

Every file (or folder) that has a name starting with . is hidden.

To list all files (including hidden ones) inside the tmp directory you just created use the **-a**(all) option of **ls**:

```
$ ls -a tmp/  
.  ..
```

CREATING EMPTY FILES

Create the ~/tmp/to-delete.txt file, using **touch**:

```
$ touch ~/tmp/to-delete.txt
```

The previous command will create the empty file. Then follow the steps below:



DELETING FILES

Delete the file you just created with **rm**:

```
$ rm ~/tmp/to-delete.txt
```



COPYING FILES

We will use the directory `/var/cadets/<location>/` in the bootcamp server to share files with our beloved `<Code Cadets>`. `<location>` is a placeholder for your bootcamp's location (eg. lisboa, fundao).

Use the **cp** command to copy the `cheat-sheet.txt` file from the shared folder into `warmup/unix/` in your home folder:

```
$ cd ~  
$ mkdir -p warmup/unix/  
$ cp /var/cadets/warmup/unix/cheat-sheet.txt ~/warmup/unix/cheat-sheet.txt
```

Make sure the file has in fact been copied to the correct folder, by listing the folder:

```
$ ls warmup/unix/  
cheat-sheet.txt
```



WHICH

As you know, in UNIX everything is a file. The commands we type on the command line (which are not part of the **bash** shell) are no exception.

The **which** command identifies the location of executables. Locate the binary of the **cp** and **which** commands:

```
$ which cp
/bin/cp

$ which which
/usr/bin/which
```

GETTING HELP

The shell has a couple of commands that can be very helpful (don't lean solely on google >> use the shell) and so are definitively worth knowing about.

When you are not sure about the name of a command (or if it exists), use **apropos** to search for a keyword.

Let's try finding a command for counting the number of words in our ~/warmup/unix/cheat-sheet.txt (there is a command for everything):

```
$ apropos count
...
wc (1)          - print newline, word, and byte counts for each file
```

We then use **man** to get familiar with the command usage and use it to our purpose:

```
$ man wc
wc [OPTION]... [FILE]...
$ wc ~/warmup/unix/cheat-sheet.txt
177 1460 8638 /home/campelo/warmup/unix/cheat-sheet.txt
```

SECURE COPY

scp copies files over a secure, encrypted network connection.

If you are familiar with **cp**, **scp** is easy to understand. Both commands require a source and a destination filesystem location. The big difference is that with **scp**, one or both of the locations are on a remote server.

Lets use **scp** to send a vim cheat sheet file to our bootcamp server. First locate and download you personal favorite cheat sheet file from the web.

The **scp** command will then look something like this
(do yourself a favor and use tab completion):

```
# on the remote server
// create the destination folder
$ mkdir ~/warmup/vim/

# on a local shell session
// scp command syntax:
// $ scp source user@host:destination
$ scp ~/vim/cheat-sheet.txt campelo@lisboa.codecadets.academiadecodigo.org:~/warmup/vim/cheat-sheet.txt
cheat-sheet.txt                                100% 2957    2.9KB/s   00:00
```

UNIX ENVIRONMENT

An important UNIX concept is the **environment**, which is defined by **environment variables**. Some environment variables are set by the system, others by you, yet others by the shell, or any program that loads another program.

A variable is a character string to which we assign a value. The value assigned could be a number, text, filename, device, or any other type of data.

VARIABLES

Set your first UNIX environment variable and print its value using **echo**:

```
$ TEST="UNIX tutorial"  
$ echo $TEST  
UNIX tutorial
```

Note that environment variables are set without using \$ sign but while accessing them we use \$ sign as a prefix. These variables retain their values until we come out of the shell.

When you login to the system, the shell undergoes a phase called initialization to set up the environment. This is usually a two step process that involves the shell reading the following files:

- /etc/profile
- ~/.profile

SHELL INITIALIZATION

These are the steps involved in the shell initialization:

- The shell checks to see whether the file `/etc/profile` exists;
- If it exists, the shell reads it. Otherwise, this file is skipped and no error message is displayed;
- The shell checks to see whether the file `.profile` exists in your home directory;
- If it exists, the shell reads it; otherwise, the shell skips it and no error message is displayed.

As soon as these steps are done, the shell displays the command line prompt:

```
$
```

PROFILE FILES

The file **/etc/profile** is maintained by the system administrator of your UNIX machine and contains shell initialization information required by all users on a system.

The file **.profile** is under your control. You can add as much shell customization information as you want to this file. The minimum set of information that you need to configure includes:

- The type of terminal you are using;
- A list of directories in which to locate commands;
- A list of variables affecting the look and feel of your terminal.

Check your **.profile** using **vim**:

```
$ vim ~/.profile
```

PATH VARIABLE

The **PATH** variable specifies the location in which the shell looks for commands.

Try printing your **PATH** variable in the console:

```
$ echo $PATH  
/usr/local/bin:/usr/bin:/bin:/usr/local/games:/usr/games
```

PS1 VARIABLE

The characters that the shell displays as your command prompt are stored in an environment variable called PS1. You can change this variable to be anything you want. As soon as you change it, it'll be used in the current shell session from that point on.

Try changing the PS1 to the following:

```
$ PS1="\u@\h \w\$ "  
campelo@guincho ~$
```

If all went well, the command prompt of your current shell session has become something like `campelo@guincho ~$`.

Now make that change load automatically when the shell initializes by adding the PS1 variable to `~/.profile`.

```
campelo@guincho ~$ vim ~/.profile
```



FILESYSTEM SECURITY (ACCESS RIGHTS)

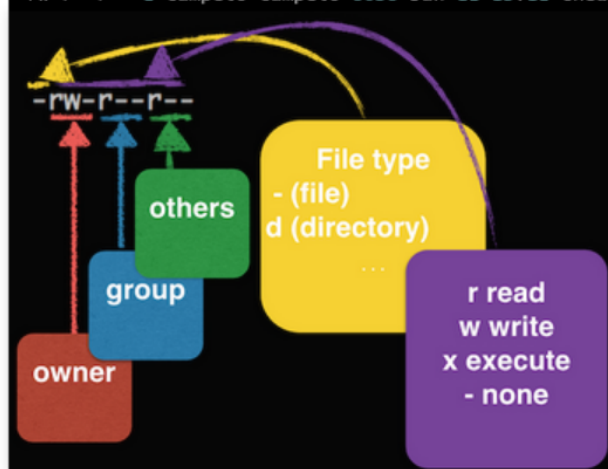
Access rights is an important component of UNIX that provides a secure method for storing files. Every file in UNIX has the following attributes:

- Owner permissions: The owner's permissions determine what actions the owner of the file can perform on the file;
- Group permissions: The group's permissions determine what actions a user, who is a member of the group that a file belongs to, can perform on the file;
- Other (world) permissions: The permissions for others indicate what action all other users can perform on the file.

FILE PERMISSIONS

Try long listing the unix cheat-sheet.txt file you copied before:

```
campelo@guincho:~$ cd warmup/unix/  
campelo@guincho:~/warmup/unix$ ls -l cheat-sheet.txt  
-rw-r--r-- 1 campelo campelo 8638 Jan 11 15:11 cheat-sheet.txt
```



Here the first column (with the exception of the first character) represents different access modes (permissions) associated with the file.

The permissions are broken into groups of threes. Each position in a group denotes a specific permission. In this order, read (r), write(w) and execute(x).